

Ask Safely: Privacy-Aware LLM Query Generation for Knowledge Graphs*

Mauro Dalle Lucca Tosi

mauro.dalle-lucca-tosi@list.lu

Luxembourg Institute of Science and Technology (LIST)
Esch-sur-Alzette, Luxembourg

Jordi Cabot

jordi.cabot@list.lu

Luxembourg Institute of Science and Technology (LIST)
University of Luxembourg (UNILU)
Esch-sur-Alzette, Luxembourg

Abstract

Large Language Models (LLMs) are increasingly used to query knowledge graphs (KGs) due to their strong semantic understanding and extrapolation capabilities compared to traditional approaches. However, when KGs contain sensitive information and users lack local access to generative models, privacy becomes a critical concern. To address this issue, we propose a privacy-aware query generation approach for KGs. Our method identifies sensitive information in the graph based on its structure and omits such values before requesting the LLM to translate natural language questions into Cypher queries. Experimental results show that our approach effectively prevents sensitive data from being transmitted to third-party services, while maintaining a high level of query accuracy.

CCS Concepts

• **Information systems** → **Question answering**; • **Computing methodologies** → *Natural language processing*; • **Security and privacy** → **Domain-specific security and privacy architectures**.

Keywords

Question Answer, Privacy, Sensitive Data, Knowledge Graph, Q&A, Large Language Models, LLM, CYPHER

1 Introduction

Large Language Models (LLMs) are increasingly used to retrieve knowledge. However, their responses may be inaccurate or contain hallucinations. To mitigate this, many applications rely on Retrieval-Augmented Generation (RAG), especially when retrieving knowledge from documents. Extending RAG-style retrieval to Knowledge Graphs (KGs) requires providing the model with contextual graph data — a process that can compromise privacy when the KG or the user’s query includes sensitive information [12].

This poses challenges when users want to query KGs but lack the resources to deploy their own generative models, and when the question, the KG, or both involve sensitive data. Such situations are common in practice, for example in healthcare [20], biomedical [19], or legal [14] use cases, where privacy is critical. In these cases, existing approaches that transmit graph data to external LLMs are not satisfactory.

Before generative LLMs, several rule-based [21] and classical Machine Learning (ML) [3, 7] methods were developed to query KGs locally, without exposing data to third parties. These systems typically translated natural language questions into formal query languages. However, they were usually supervised and thus inherited well-known limitations: (1) the need for large training datasets, which are often unavailable; (2) poor generalization to questions that differ slightly from the training set; and (3) high adaptation costs for new use cases.

Motivated by these challenges, this paper investigates a privacy-preserving approach that leverages the semantic capabilities of LLMs without disclosing sensitive information. Given a property graph as input and without requiring additional training data, our method automatically: (1) extracts the graph schema (node labels, node properties, relation labels, and relation types); (2) builds a dictionary of sensitive values from node and relation properties (which can be manually refined by users); and (3) trains a Named Entity Recognition (NER) system to recognize the identified labels, properties, and values.

As illustrated in Figure 1, natural language questions provided by the user are then reformulated to improve clarity with respect to the graph schema, while sensitive values are masked. The LLM receives this reformulated question and is asked to generate the corresponding Cypher [10] query based solely on the graph schema passed as context. Finally, placeholders in the generated query are replaced with the original sensitive values. An additional benefit of passing only the graph schema is that it significantly reduces the number of tokens needed, which both lowers cost and improves efficiency. Moreover, the schema can be restricted to the Role-Based Access Control (RBAC) [9] sub-schema authorized for the user, ensuring that generated queries never target parts of the graph beyond user’s access rights. The resulting query can be returned to the user or executed on the graph, with the results made available accordingly.

To evaluate this privacy-aware query generation approach, we conduct an ablation study examining: (1) the effect of improving question clarity through schema information; and (2) the impact of masking sensitive data before query generation. In addition, we perform a privacy analysis to quantify both empirical and theoretical privacy gains.

2 Related Works

We divide KG-based question answering (Q&A) methods into four categories: (1) rule-based; (2) classical machine learning (ML); (3) transformer-based; and (4) LLM-based.

*This version of the article has been accepted for publication at the 20th International Conference on Research Challenges in Information Science (RCIS 2026), after peer review and is subject to Springer Nature’s AM terms of use, but is not the Version of Record and does not reflect post-acceptance improvements, or any corrections. The Version of Record is available online at: https://doi.org/10.1007/978-3-032-26836-5_4

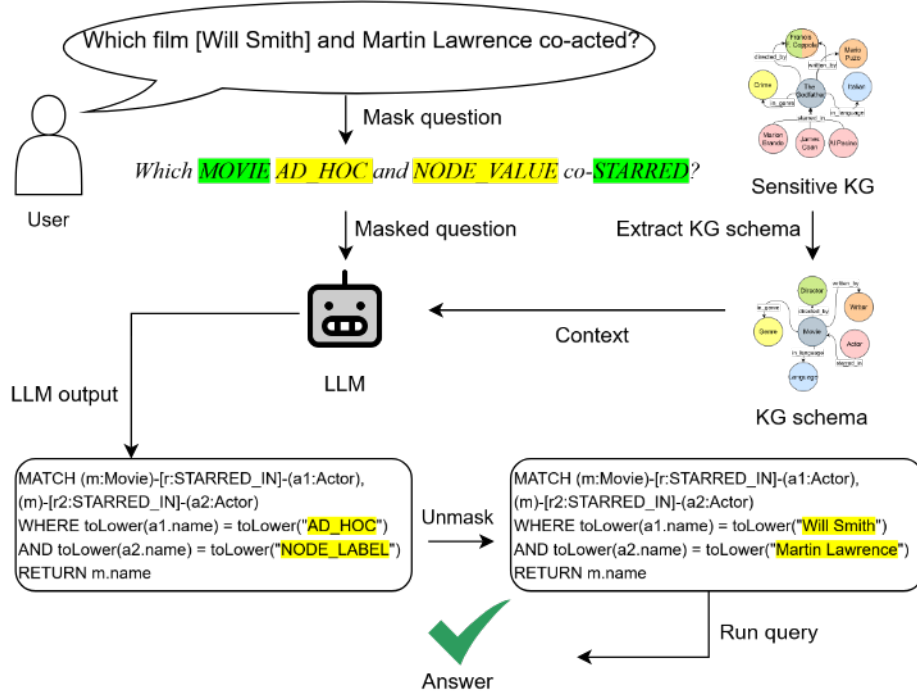


Figure 1: Workflow of the AskSafely methodology for generating queries with LLMs without disclosing sensitive information.

Rule-based methods rely on predefined rules to map natural language questions to KG query languages such as SPARQL and Cypher. They are deterministic and do not involve machine learning algorithms. For example, [21] proposes a chatbot for querying academic data, where user intent is recognized through predefined rules and managed by a finite-state system. Such methods do not require training data and provide interpretable outputs. However, they are strictly limited to the rules defined in advance, which constrains their applicability in practice.

Classical ML methods also rely on predefined rules but incorporate machine learning algorithms such as SVM [3], XGBoost, and Maximum Entropy Markov Models [7]. These approaches learn mappings from natural language to SPARQL or Cypher using training examples. While they outperform rule-based methods, they require the acquisition and preparation of training data, which introduces additional costs.

Transformer-based methods leverage the transformer architecture [28] to improve contextualization and generalization. Those methods — such as [2], [23], and [25] — typically rely on BERT variants fine-tuned on examples of expected user queries. This line of work has substantially improved the accuracy of KG Q&A and continues to achieve state-of-the-art results [24]. However, existing approaches are usually developed for specific knowledge graphs, which limits their transferability. Applying them to new domains requires significant effort to design specialized solutions and to collect and clean large datasets of training queries.

LLM-based methods rely on generative large language models such as GPT [1], LLaMA [8], and DeepSeek [15]. These approaches achieve excellent performance [24] thanks to their generalization

and contextualization capabilities, often without requiring training data. In fact, their strongest results are obtained via zero-shot prompting [30]. Furthermore, recent studies [18, 22] have shown that LLMs can generate accurate queries even when provided solely with the graph schema as contextual information. However, LLMs are prone to hallucinations, which must be carefully mitigated. Moreover, due to their large number of parameters, hosting state-of-the-art LLMs locally is impractical for most users. Consequently, applications typically rely on third-party services provided by companies such as OpenAI, Google, or DeepSeek AI.

In scenarios where labeled training data is scarce or nonexistent and where sensitive data is involved, users are left with two imperfect options: rule-based systems or classical ML methods that require less training data. Yet, as noted above, both categories have limited semantic ability to interpret user questions, since anything not explicitly covered by predefined rules or training examples often produces suboptimal results. Therefore, developing an approach that enables effective Q&A over KGs while addressing the privacy concerns of relying on third-party generative LLMs remains an open research problem. Table 1 summarizes the characteristics of existing KG-based Q&A methods and highlights how our proposed privacy-aware LLM approach compares.

3 Privacy-Aware Q&A

In this paper, we investigate how to leverage the contextualization and generalization capabilities of LLMs to automatically query sensitive data from KGs using natural language questions, which may themselves contain sensitive information. We propose *AskSafely*, a privacy-aware query generation method that enables users to rely

KG-based Q&A	No additional training/data	Generalization	Privacy
Rule-based	X		X
Classical ML		X	X
Transformers-based		X	X
LLM-based	X	X	
Privacy-aware LLM (our method)	X	X	X

Table 1: Comparison of KG-based Q&A methods, with “X” marking characteristics those methods have.

on third-party generative LLM services without sharing sensitive data — critical in scenarios constrained by organizational policies, ethical standards, non-disclosure agreements, or legal frameworks. Throughout this paper, we illustrate our method using the METAQA dataset [29], which, although it does not contain sensitive data, is treated as if all values in the KG were sensitive.

When generating KG queries from natural language questions using LLMs, there are two main sources of potential sensitive data leakage. First, the contextual information provided to the LLM, which often includes the data from the KG itself [4–6, 16, 26]. Second, the user’s question may contain sensitive information in its text; for example “Other than Bad Boys, which movies did Will Smith and Martin Lawrence co-star in?” implies prior collaboration between the actors, which could be considered private information.

Below, we describe how our method addresses each of these challenges.

3.1 Graph schema as context

Our method relies on the observation that KGs used for Q&A tasks are typically based on an ontology or predefined schema. Figure 2 illustrates this concept.

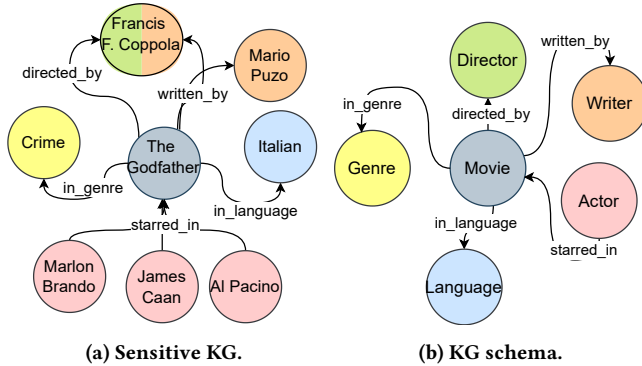


Figure 2: “The Godfather” sub-KG and its schema.

Figure 2a shows a KG containing seven nodes connected to the movie “The Godfather”. This graph is constructed based on the schema in Figure 2b, which includes six node types: “Movie”, “Writer”, “Director”, “Genre”, “Language”, and “Actor”. The schema dictates node types, their attributes, and relationships. While the data stored in the KG may be sensitive, the schema itself is not, and

can therefore provide sufficient context for the LLM to generate executable queries without accessing sensitive values. In addition to preserving privacy, providing only the graph schema substantially reduces the prompt size. This improves token efficiency, which directly translates into lower costs and faster query generation.

Moreover, because the schema can be restricted to the sub-schema that the user is authorized to access, the LLM is prevented from generating queries that target parts of the KG beyond their access rights. This integrates access control directly into the query-generation pipeline, ensuring that privacy is preserved not only at the level of sensitive values, but also at the level of graph topology.

In detail, we represent the schema of a knowledge graph using its fundamental components:

- Node labels (NODE_LABEL), which define the entity types represented in the graph;
- Node properties (NODE_PROPERTY) and node values (NODE_VALUE), describing the properties of nodes and their corresponding instances;
- Relation labels (RELATION_LABEL), which specify the types of edges linking nodes;
- Relation properties (RELATION_PROPERTY) and relation values (RELATION_VALUE), describing the properties of relations and their corresponding instances;

As illustrated in Figure 2, the label “Director” is a NODE_LABEL, one could infer the existence of “name” as a NODE_PROPERTY, and “Francis F. Coppola” corresponds to a NODE_VALUE. The relation “directed_by” represents a RELATION_LABEL connecting the movie node to the director node. In this example, relations have no additional properties or values, but if present, they would follow the same representation pattern as those defined for nodes.

3.2 Masking user questions

User questions may contain sensitive information and domain-specific vocabulary that is difficult for the LLM to interpret when the context given omits private information. To address this, we identify two sets of entities: (i) sensitive entities that must not be sent directly to the LLM, and (ii) key entities critical for query generation, representing node labels, relations, and attributes.

For example, consider the question below, which has sensitive data explicitly flagged between brackets.

- **Sensitive question:**

Which film [Will Smith] and Martin Lawrence co-acted?

Based on the graph schema shown in Figure 2b, we identify the following fundamental components of the KG as entities: NODE_LABEL, NODE_PROPERTY, NODE_VALUE, RELATION_LABEL, RELATION_PROPERTY, and RELATION_VALUE. Additionally, we introduce the AD_HOC entity type, which represents user-specified sensitive values not explicitly present in the KG. These entities are then masked to ensure that no sensitive information is sent to the LLM.

- **Pre-processed question:**

Which NODE_LABEL AD_HOC and NODE_VALUE co-RELATION_LABEL?

Next, key entities are substituted with their canonical KG synonyms to reduce the risk of the LLM using incorrect terminology. For instance, “film” is replaced by “MOVIE” and “acted” by “STARRED”, matching the KG labels and properties. The resulting privacy-aware question submitted to the LLM is:

- Privacy-aware question:

Which MOVIE AD_HOC and NODE_VALUE co-STARRED_IN?

Key and sensitive entities can be identified using a NER algorithm appropriate to the user’s domain. In our implementation, this step is performed using a simple rule-based approach with case-insensitive matching between question tokens and KG entities, including their manually defined synonyms. More advanced NER models could also be used in practice.

3.3 Privacy-aware LLM prompt and reply

Once the KG context and user question are free from sensitive information, we construct a zero-shot prompt instructing the LLM to generate a Cypher query to retrieve the requested information. Because the prompt includes only schema-level information rather than the entire graph, it remains compact and token-efficient, allowing us to scale to larger KGs without overwhelming the LLM input. The prompt we use throughout our experiments can be visualized in Appendix 1.

As an example, the LLM’s response to the privacy-aware question described in Section 3.2 is shown below:

- LLM reply:

```
MATCH (m:Movie)-[r:STARRED_IN]-(a1:Actor),
(m)-[r2:STARRED_IN]-(a2:Actor)
WHERE toLower(a1.name) = toLower("AD_HOC")
AND toLower(a2.name) = toLower("NODE_LABEL")
RETURN m.name
```

Before execution, placeholders are replaced with the original sensitive values provided by the user, yielding the final query:

- Final Cypher query:

```
MATCH (m:Movie)-[r:STARRED_IN]-(a1:Actor),
(m)-[r2:STARRED_IN]-(a2:Actor)
WHERE toLower(a1.name) = toLower("Will Smith")
AND toLower(a2.name) = toLower("Martin Lawrence")
RETURN m.name
```

Finally, the Cypher query is executed, and the results returned.

4 Experiments & Results

Considering that the capability of LLMs to accurately generate Cypher queries using the graph schema has already been demonstrated [18, 22], our objective is twofold: (1) to investigate the impact of applying the *AskSafely* privacy constraints on the quality of the queries generated by these models, and (2) to assess both the practical and theoretical privacy gains achieved through the use of *AskSafely*.

To implement the privacy-aware strategy for querying KGs described in Section 3, we used the BESSER Agentic Framework (BAF) [11]. BAF is designed to facilitate and accelerate the development of AI agents in a low-code manner. It natively implements the

simple NER method described previously and allows the definition of synonyms, which are automatically used to identify key entities. Additionally, BAF integrates generative-LLM APIs, enabling seamless use of LLMs during agent execution. All the code used in our experiments is available on GitHub¹.

Dataset: We evaluate our strategy using the METAQA dataset [29], which contains facts for 16,427 movies, including their actors, directors, writers, release years, language, tags, genre, IMDB votes, and IMDB ratings. The dataset also provides natural language questions and corresponding answers, with entities already annotated between brackets. We use the “vanilla” test set, containing 39,093 questions. Most of these questions follow the same pattern, varying only in entity values (e.g., actor or movie names). Because these values are automatically replaced in our privacy-aware strategy, we reduced the dataset to 503 questions with unique patterns. A k-hop query refers to a question that requires traversing k relations in the knowledge graph to reach the answer. The METAQA dataset includes questions requiring one-, two-, and three-hop reasoning, making it a challenging benchmark that covers both simple and complex reasoning tasks. Among the 503 unique questions, 143 require 1-hop, 210 require 2-hop, and 150 require 3-hop queries to correctly answer the question.

4.1 Experimental setting

First, we created a property graph in Neo4J [13] using the triples from METAQA. The graph follows the schema illustrated in Figure 2b. Each node has a “name” property, and “Movie” nodes also include properties corresponding to their connected entities and movie-specific attributes: “release year”, “tags”, “IMDB votes”, and “IMDB ratings”.

Next, we extract from the graph and provide to BAF the node types, relations, properties, property values, and their synonyms. In our implementation, most steps of the pipeline are automatic. The graph schema and property values are automatically extracted from Neo4J, and masking, synonym substitution, prompt construction, query generation, and query execution are performed automatically by BAF and the LLM. The only manual configuration concerns the definition of synonyms and the specification of which entity values are considered sensitive — when all values in the graph are treated as sensitive, this configuration step can also be automated.

For illustration, we manually defined a few synonyms, shown in Table 2. In practice, most synonyms for non-sensitive entities (e.g., node or relation labels) can be automatically suggested by the LLM itself, as they do not contain private information. In contrast, sensitive entities — such as property values corresponding to personal names or other private data — require special attention. Synonyms for these entities should not be generated by the LLM but instead obtained from a controlled external list or through a domain-adapted NER algorithm, depending on the specific use case. Nonetheless, such synonyms are primarily used to enhance the entity recognition process and are not strictly required for our approach to function effectively.

We then define which entities are sensitive. For this experiment, all property values in the KG are considered sensitive, except for single-word movie titles and movie tags. Terms like “Creator”, “Fall”,

¹<https://github.com/maurodl/PrivateNL2CYPHER>

Entity	Synonyms
Movie	film, movie, films, movies
directed_by	directed
in_language	language
release_year	year, release, released
has_tags	tags, tag, described, about
written_by	wrote, written
starred_actors	actor, actress, actors, actresses, star, starred
has_imdb_votes	votes, vote
has_genre	genre, type
has_imdb_rating	rating
name	names

Table 2: List of synonyms defined in BAF.

and “Primary” (movie titles) or “main” and “script” (tags) are not considered sensitive, as they are unlikely to reveal private information and are more likely to appear as common words in contexts unrelated to movies or tags.

4.2 Q&A Performance

We evaluated our agent on the 503 selected questions and compared the generated query results with the corresponding dataset answers. Table 3 presents the accuracy obtained under three configurations: (1) GPT-based query generation without privacy constraints, (2) our privacy-aware approach without synonym substitution, and (3) our full privacy-aware approach with synonym substitution. To better understand how question complexity affects performance, results are additionally separated according to the number of hops (1-hop, 2-hop, and 3-hop) required to answer each question. For each configuration, both the automatically computed accuracy and the manually verified accuracy are reported. The manual evaluation was necessary because the dataset’s reference answers were sometimes incomplete. For instance, for the question about the genre of the movie “Bad Boys”, the dataset expects [“Drama”, “Comedy”], even though the underlying knowledge graph includes additional genres such as [“Drama”, “Comedy”, “Crime”, “Action”].

By substituting the sensitive tokens present in the users’ questions by placeholders, we avoid leakage of sensitive data to third party companies. Still, to guarantee the increase in privacy in those setting we perform 3 analysis on the privacy gained when using *AskSafely*. First, we track if and how many sensitive tokens were exposed during our experiments. Second, we test the possibility of indirect leakage of sensitive information. Third, we calculate the reduction of mutual information between the users question and sensitive tokens before and after the masking.

4.2.1 Token Exposure. We calculate the total number of sensitive tokens exposed during the executing of *AskSafely* in the MetaQA dataset. Considering that the dataset already flags sensitive tokens, as expected, *AskSafely* had 0 tokens exposed during the experiments.

4.2.2 Leakage Test. We empirically evaluate if it was possible to infer sensitive tokens from the remaining context passed to the LLMs. For this, we asked *gpt5-mini* to guess the masked values based on

the context previously given to the LLM via the prompt in Appendix 2.

After manually comparing the LLM guesses with the sensitive values contained in the original users’ questions, we observed that the LLM could not make a single correct guess. In general, the LLM mostly just guessed the name of famous actors such as Tom Hanks, famous directors such as Christopher Nolan, and famous movies such as The Matrix, and Inception.

4.2.3 Mutual Information Analysis. Beyond the empirical leakage test, we further estimate how much information about the sensitive values may still be inferred from the transmitted data. To this end, we interpret mutual information $I(X; Z)$ as a measure of how much dependence remains between sensitive inputs and observable outputs [27], considering the sensitive entities X and the users’ questions, both in their masked form Z and in their original form Z' :

$$I(X; Z) = \sum_{z \in Z} \sum_{x \in X} P_{(X,Z)}(x, z) \log \left(\frac{P_{(X,Z)}(x, z)}{P_X(x)P_Z(z)} \right). \quad (1)$$

Since we work with textual data, estimating exact probability distributions is not feasible. Instead, we approximate them using the distribution of embedding similarities. Specifically, we derive pseudo-probabilities from cosine similarities between the embeddings of X and Z , normalized through a softmax transformation:

$$I(X; Z) \approx \sum_{i,j} \tilde{p}(x_i, z_j) \log \left(\frac{\tilde{p}(x_i, z_j)}{\tilde{p}_X(x_i)\tilde{p}_Z(z_j)} \right), \quad (2)$$

$$\text{with } \tilde{p}(x_i, z_j) = \frac{\exp(\cos(f(x_i), f(z_j)))}{\sum_{k,l} \exp(\cos(f(x_k), f(z_l)))}.$$

Finally, we quantify the privacy improvement achieved by masking through the relative reduction in mutual information:

$$\text{Privacy gain} = \frac{I(X, Z) - I(X, Z')}{I(X, Z)} \times 100\% = \frac{0.0024 - 0.0007}{0.0024} = 71.1\%. \quad (3)$$

Despite the already low initial mutual information between user questions and sensitive values, these results show that *AskSafely* further reduces potential information leakage by approximately 71%.

5 Discussion

In this section, we discuss the key insights gained from the experiments described in Section 4.

- **Privacy-awareness impact on performance.** Table 3 shows that GPT maintains consistently high accuracy across questions of different complexity levels. In the privacy-aware setting, accuracy decreases as the number of hops increases, which we attribute to the reduced contextual information available to the LLM after masking sensitive tokens. Nevertheless, synonym substitution recovers a substantial portion of this loss and improves performance across all hop levels. Interestingly, in the case of simple 1-hop questions, the privacy-aware approach with synonym substitution even slightly improves performance compared to the baseline, suggesting that aligning user vocabulary with the schema can help guide the query generation process.

Method	1-Hop		2-Hop		3-Hop		Overall	
	Acc	M-Acc	Acc	M-Acc	Acc	M-Acc	Acc	M-Acc
GPT-5 (no privacy)	81.0	90.1	92.9	97.7	74.0	92.7	83.9	94.0
Privacy-aware (no synonym)	85.9	95.1	78.6	82.9	55.3	62.7	73.7	80.3
Privacy-aware (complete)	91.5	97.2	85.7	89.6	63.3	73.3	80.7	86.9

Table 3: Accuracy of GPT-5 with and without the proposed privacy-aware methodology, separated by the number of hops required to answer each question. Acc denotes automatically computed accuracy, while M-Acc denotes manually verified accuracy. The final columns report overall results across all questions.

- **Privacy and scalability are not trade-offs.** By transmitting only the graph schema, our approach not only protects sensitive information but also keeps the LLM prompt compact. In our experiments, each LLM query required fewer than 1,400 tokens in total, with less than 500 tokens used to represent the METAQA schema. Based on this efficiency, we estimate that *AskSafely* can scale to knowledge graphs containing approximately 800× more node types and relations than those evaluated in Section 4 – that is, around 4,800 node types and 4,000 relation types. It is important to note that this estimate concerns the *schema* of the graph, not its instance size; in principle, *AskSafely* imposes no limit on the number of nodes or edges in the underlying KG. These findings demonstrate that privacy preservation and computational efficiency are not competing objectives but rather complementary ones.
- **Choosing synonyms.** The importance of defining synonyms can be seen in Table 3. In our experiments, most of the accuracy gain when substituting synonyms came from replacing the word “about” with “has_tags.” While this substitution may sound unnatural in natural language, it matched the way the dataset was structured (e.g., mapping “what topics is [Free Willy] about” to movie tags). In practice, for questions following a pattern closer to natural language than the one provided by the MetaQA dataset, we expect synonyms automatically identified by an LLM to perform satisfactorily. Still, if a background ontology of the domain in question is available, their pre-defined synonyms could further improve *AskSafely* accuracy.
- **LLMs also make mistakes.** The most frequent issue observed was confusion between entity labels. For example, the question “who is listed as director of the movies starred by [Solo] actors” was misinterpreted, with the LLM treating “Solo” as the name of an actor rather than the name of a movie, leading to incorrect results. Such error was more present in *AskSafely* than pure GPT-5 due to the reduction of semantic information given to the LLM, which depending on the way the question was formulated, did not suffice for it to produce an accurate query. Another source of errors was returning more information than requested. For instance, when asked “when were the films directed by

[Peter Lord] released”, the system returned [‘Chicken Run’, 2000]” instead of the expected “[2000]”.

- **Supervised Methods and Generalization Challenges.** [17] report 100% accuracy on the MetaQA dataset by training a sequence-to-sequence transformer model to map natural language questions into graph paths. These paths are output as Prolog functions, which can then be executed to retrieve the correct answers. However, this approach is entirely dependent on training data and lacks generalization. Small variations in phrasing that were not seen during training consistently failed. For example, questions such as “Which movies starred [Al Pacino]?”, “Which movies had [Al Pacino] as an actor?”, “Which movies featured [Al Pacino]?”, or “Which movies have [Al Pacino]?” all produced incorrect results. After several attempts – with the query “Which movies [Al Pacino] starred in?” – the model was able to generate the correct answer. Thus, we emphasize the need to investigate the generalization capabilities of supervised methods, especially in Q&A settings where users are unlikely to phrase their questions according to a predefined schema.

5.1 Limitations

Below we highlight the following limitations we observed while developing and testing *AskSafely*.

- (1) **Not suitable for anonymous graph schemas.** If the schema of the graph contains sensitive information, such as a trade secret, *AskSafely* cannot be used by design.
- (2) **Indirect leakage of masked terms.** Despite the privacy analysis presented in Section 4.2.2, we cannot fully guarantee that no sensitive information can be inferred from masked questions. Such inference largely depends on how the question is formulated and on the contextual information available.
- (3) **Domain dependent NER.** The choice of the NER algorithm depends on the domain, and direct word matching should be avoided in practice.
- (4) **Synonyms definition.** Synonyms for sensitive values must be defined according to the specific use case, which may be impractical for large knowledge graphs.

Nevertheless, these limitations are unlikely to affect the applicability of *AskSafely* in most scenarios. This claim is supported by the following observations:

- Graph schemas seldom contain sensitive information;
- The leakage tests can be easily adapted to different domains using representative samples of expected question types;
- The same NER tools and thesauri or ontologies used to build or maintain the knowledge graph can also be leveraged by *AskSafely*. When domain-specific resources are unavailable, generic ones may still yield satisfactory results, as demonstrated in Section 4.

6 Conclusion

We introduced a privacy-aware strategy for generating Cypher queries with third-party generative LLMs. By using the graph schema as context and masking sensitive information in user questions, our approach preserves privacy while still benefiting from the strong generalization capabilities of LLMs. Moreover, relying only on the graph schema keeps prompts compact and token-efficient, enabling scalability to larger graphs. Experiments on the METAQA dataset achieved up to 86.9% accuracy with 0% direct sensitive information leakage and 71% reduction in potential indirect leakage, demonstrating that privacy can be protected while maintaining query quality. In future work, we aim to extend this approach to domain-specific use cases. We also plan to automatically filter and adapt the graph schema to the RBAC model provided for the users. Together, these directions will allow us to explore how privacy-preserving and access control mechanisms can be integrated with LLM-based reasoning at scale.

Acknowledgments

This project is supported by the Luxembourg National Research Fund (FNR) PEARL program, grant agreement 16544475.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [2] Sareh Aghaei, Elie Raad, and Anna Fensel. 2022. Question answering over knowledge graphs: A case study in tourism. *IEEE Access* 10 (2022), 69788–69801.
- [3] Addi Ait-Mlouk and Lili Jiang. 2020. KBot: a Knowledge graph based chatBot for natural language understanding over linked data. *IEEE Access* 8 (2020), 149220–149230.
- [4] Artem Alekseev, Mikhail Chaichuk, Miron Butko, Alexander Panchenko, Elena Tutubalina, and Oleg Somov. 2025. The benefits of query-based KGQA systems for complex and temporal questions in LLM era. In *International Conference on Applications of Natural Language to Information Systems*. Springer, 426–441.
- [5] Tu Ao, Yanhua Yu, Yuling Wang, Yang Deng, Zirui Guo, Liang Pang, Pinghui Wang, Tat-Seng Chua, Xiao Zhang, and Zhen Cai. 2025. Lightprof: A light-weight reasoning framework for large language model on knowledge graph. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 23424–23432.
- [6] Caio Viktor S Avila, Marco A Casanova, and Vânia MP Vidal. 2024. A framework for question answering on knowledge graphs using large language models. In *European Semantic Web Conference*. Springer, 168–172.
- [7] Yi-Hui Chen, Eric Jui-Lin Lu, and Ting-An Ou. 2021. Intelligent SPARQL query generation for natural language processing systems. *IEEE Access* 9 (2021), 158638–158650.
- [8] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. 2024. The llama 3 herd of models. *arXiv e-prints* (2024), arXiv-2407.
- [9] David Ferraiolo, Janet Cugini, D Richard Kuhn, et al. 1995. Role-based access control (RBAC): Features and motivations. In *Proceedings of 11th annual computer security application conference*. 241–48.
- [10] Nadime Francis, Alastair Green, Paolo Guagliardo, Leonid Libkin, Tobias Lindaker, Victor Marsault, Stefan Plantikow, Mats Rydberg, Petra Selmer, and Andrés Taylor. 2018. Cypher: An evolving query language for property graphs. In *Proceedings of the 2018 international conference on management of data*. 1433–1445.
- [11] Marcos Gomez-Vazquez, Aaron Conrardy, and Jordi Cabot. 2025. *BESSER Agentic Framework (BAF)*. <https://github.com/BESSER-PEARL/BESSER-Agentic-Framework>
- [12] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. 2025. A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions. *ACM Trans. Inf. Syst.* 43, 2, Article 42 (Jan. 2025), 55 pages. doi:10.1145/3703155
- [13] Neo4j Inc. 2025. *Neo4j*. <https://neo4j.com>
- [14] Jinqi Lai, Wensheng Gan, Jiayang Wu, Zhenlian Qi, and Philip S. Yu. 2024. Large language models in law: A survey. *AI Open* 5 (2024), 181–196. doi:10.1016/j.aiopen.2024.09.002
- [15] Aixian Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2024).
- [16] Jie Ma, Zhitao Gao, Qi Chai, Wangchun Sun, Pinghui Wang, Hongbin Pei, Jing Tao, Lingyun Song, Jun Liu, Chen Zhang, et al. 2025. Debate on graph: a flexible and reliable reasoning framework for large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 24768–24776.
- [17] Navid Madani and Kenneth Joseph. 2023. Answering Questions Over Knowledge Graphs Using Logic Programming Along with Language Models. In *Tiny papers@ ICLR*.
- [18] Ioanna Mandilara, Christina Maria Androna, Eleni Fotopoulou, Anastasios Zafeiropoulos, and Symeon Papavassiliou. 2025. Decoding the Mystery: How Can LLMs Turn Text Into Cypher in Complex Knowledge Graphs? *IEEE Access* (2025).
- [19] Nicholas Matsumoto, Jay Moran, Hyunjun Choi, Miguel E Hernandez, Mythreye Venkatesan, Paul Wang, and Jason H Moore. 2024. KRAGEN: a knowledge graph-enhanced RAG framework for biomedical problem solving using large language models. *Bioinformatics* 40, 6 (06 2024), btae353.
- [20] Richard May and Kerstin Denecke. 2022. Security, privacy, and healthcare-related conversational agents: a scoping review. *Informatics for Health and Social Care* 47, 2 (2022), 194–210. doi:10.1080/17538157.2021.1983578 PMID: 34617857.
- [21] Antonello Meloni, Simone Angioni, Angelo Salatino, Francesco Osborne, Diego Reforgiato Recupero, and Enrico Motta. 2023. Integrating conversational agents and knowledge graphs within the scholarly domain. *Ieee Access* 11 (2023), 22468–22489.
- [22] Akseli Nuutila. 2025. *Evaluating LLM-Based Cypher Query Generation from Natural Language over a CPQ Data Knowledge Graph*. Master’s thesis. University of Turku. Available at <https://urn.fi/URN:NBN:fi-fe20251216120610>.
- [23] Trung Phan Hong and Phuc Do. 2025. A Novel Multi-hop Query Answering System Based on a Large Knowledge Graph and Distributed Computing. *ACM Transactions on Asian and Low-Resource Language Information Processing* 24, 3 (2025), 1–28.
- [24] Phillip Schneider, Manuel Klettner, Kristina Jokinen, Elena Simperl, and Florian Matthes. 2024. Evaluating Large Language Models in Semantic Parsing for Conversational Question Answering over Knowledge Graphs. In *Proceedings of the 16th International Conference on Agents and Artificial Intelligence*. SCITEPRESS-Science and Technology Publications.
- [25] Xiaqiang Tang, Jian Li, Nan Du, and Sihong Xie. 2025. Adapting to non-stationary environments: Multi-armed bandit enhanced retrieval-augmented generation on knowledge graphs. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 12658–12666.
- [26] Shiyu Tian, Shuyue Xing, Xingrui Li, Yangyang Luo, Caixia Yuan, Wei Chen, Huixing Jiang, and Xiaojie Wang. 2025. A systematic exploration of knowledge graph alignment with large language models in retrieval augmented generation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 25291–25299.
- [27] Ayşe Ünsal and Melek Önen. 2023. Information-theoretic approaches to differential privacy. *Comput. Surveys* 56, 3 (2023), 1–18.
- [28] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [29] Yuyu Zhang, Hanjun Dai, Zornitsa Kozareva, Alexander J. Smola, and Le Song. 2018. Variational reasoning for question answering with knowledge graph. In *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence and Thirtieth Innovative Applications of Artificial Intelligence Conference and Eighth AAAI Symposium on Educational Advances in Artificial Intelligence* (New Orleans, Louisiana, USA) (AAAI’18/IAAI’18/EAAI’18). AAAI Press, Article 745, 8 pages.
- [30] Yuqi Zhu, Xiaohan Wang, Jing Chen, Shuofei Qiao, Yixin Ou, Yunzhi Yao, Shumin Deng, Huajun Chen, and Ningyu Zhang. 2024. Llms for knowledge graph construction and reasoning: Recent capabilities and future opportunities. *World Wide Web* 27, 5 (2024), 58.

Appendix

1 *AskSafely* Main prompt

You are a tool that transforms natural language questions into Cypher queries.
The graph structure is: {graph_structure}.

Your task: Generate a single Cypher query that best represents the question:
{masked_message}.

STRICT RULES (must always be followed):

1. Output **only** the Cypher query (no explanations, no extra text, no formatting).
2. Never include relationship types or directions.
 - Always write relationships as `~-[r]~``.
 - Forbidden: `~-[r]->``, `~<-[r]~``, `~[:RELATION_TYPE]~``.
 - Wrong: `~(a)-[r]->(b)~`` → Correct: `~(a)-[r]~(b)~``
3. Always restrict nodes by labels **as** they appear in `graph_structure``.
 - If a node label exists in `graph_structure``, you must include it (e.g. `~(m:Movie)~``, `~(w:Writer)~``).
 - Do not invent or guess labels.
 - Do not omit labels when they are defined in `graph_structure``.
4. Never return whole nodes. Always return a property and remove duplicates:
 - Example: `~RETURN DISTINCT w.name~``
5. Example of bad vs good:

```
- Bad (labels missing): MATCH (m)-[r]-(w) WHERE m.name = 'AD_HOC'
  RETURN DISTINCT w.name
- Good (labels enforced from graph structure): MATCH (m:Movie)-[r]-(w:Writer)
  WHERE toLower(m.name) = toLower('AD_HOC') RETURN DISTINCT w.name
```

Failure to follow these rules will result in an invalid answer.

2 Leakage Test Prompt

You are an assistant specialized in reconstructing anonymized elements using structural graph context.

The question you will see contains one placeholder: AD_HOC.

The only available contextual information comes from `graph_structure`, which describes the relationships or topology relevant to the question.

Your task: use `graph_structure` to infer the most plausible replacement for AD_HOC.

Output strictly: return the inferred replacement text only – no commentary, formatting, or punctuation.

Question: {masked_question}

`graph_structure`: {graph_structure}

Reply only with your guess, do not explain.